Project Work Breakdown: Custom 2D Platformer

1. Project Overview

- **Objective:** Develop a 2D platformer game based heavily on the classic Super Mario Bros (1985) gameplay using C++ with a strong emphasis on Object-Oriented Programming (OOP) principles. *Custom characters, enemies, and items will be treated as bonus features (Custom Level) if time permits.*
- **Tech Stack:** C++, SFML 2.6 for rendering and window management.
- **Core Requirements:**
    - Strict application of OOP concepts (inheritance, polymorphism, encapsulation, abstraction).
    - Implementation of at least 5 design patterns (e.g., Factory, Singleton, Observer) to ensure a modular architecture.
    - Completion of 3 distinct levels with increasing difficulty, resembling classic Mario levels.
- **Team Members:** Minh Khoa, Khải, Long, Đăng Khoa.
- **Duration:** 9 Weeks. (Note: Weeks 4 and 7 are designated as light-duty weeks, limited to 4-5 hours per member).

2. Task Division (Work Breakdown Structure)

This division ensures a balanced workload, allowing each member to handle core mechanics while securing the required design patterns.

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
Minh Khoa	**Core Engine & Physics**	Singleton, State Pattern	- Manage the SFML RenderWindow, Game Loop, and Game States. - Implement precise AABB collision detection.

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
			- Apply gravity and physics
Khải	**Hero & Items Mechanics**	Factory, Strategy Pattern	- Build the base `Character` class and the `Mario` class (and other Character class) - Implement classic items (Mushroom, Coin, Fire Flower) using the Factory Pattern. - *Bonus:* Develop custom items/mechanics as a stretch goal for custom levels.
Long	**Enemies & AI**	Factory, Template Method	- Inherit the Character class to create the base Enemy abstract class. - Create classic enemies (Goombas, Koopas). - Implement standard patrol AI using the Template Method.

Team Member	Core Responsibility	OOP & Design Patterns Focus	Granular Key Tasks
			- *Bonus:* custom enemies deferred to the bonus phase.
Đăng Khoa	**Level, UI/UX & Audio**	Observer, Singleton Pattern	- Build a Map Parser to load classic Mario levels from text/csv files. - Implement the HUD (score, coins, lives) via the Observer Pattern. - Develop a SoundManager for 8-bit BGM and SFX. - *Bonus:* Explore a custom level editor if time allows.

3. Project Timeline

Note: Regular weeks require 8-10 hours per member. Weeks 4 and 7 are limited to 4-5 hours per member.

Week 1: Foundations & Concept (8-10 hrs)

- **All Members:** Research game loop fundamentals and Core OOP principles. Finalize the

5 specific Design Patterns. Extract and gather sprites and audio assets.

Week 2: Core Skeleton & Initialization (8-10 hrs)

- **Minh Khoa:** Initialize the C++/SFML project. Implement the primary game loop and basic gravity/collision.
- **Đăng Khoa:** Develop the tilemap renderer to display a static Mario level (bricks, pipes) from a text file.
- **Khải:** Code the `Mario` class with basic keyboard inputs for movement and jumping.
- **Long:** Structure the base `Enemy` class.

Week 3: Entities & Interactions (8-10 hrs)

- **Khải:** Spawn classic items (Mushroom, Coin) and code Mario's size-changing logic (Strategy Pattern).
- **Long:** Spawn Goombas/Koopas and implement basic walking/bouncing behavior.
- **Minh Khoa:** Integrate complex collision detection (e.g., Mario stomping on Goombas).

Week 4: Refactoring & UI Integration (4-5 hrs - Light Week)

- **All Members:** Code review and OOP architecture cleanup.
- **Đăng Khoa:** Implement the Observer pattern to update the classic Mario UI (score, coins, time) dynamically.

Week 5: Core Mechanics Completion (8-10 hrs)

- **Long:** Finalize enemy logic (e.g., Koopa shells sliding and hitting other enemies).
- **Khải:** Implement invincibility frames and Fire Mario state transitions.
- **Minh Khoa:** Refine hitboxes to perfectly match the 1985 gameplay feel.

Week 6: Level Design (8-10 hrs)

- **Đăng Khoa & All Members:** Fully construct and balance the 3 required classic levels. Place enemies and items strategically.

Week 7: Playtesting & Audio (4-5 hrs - Light Week)

- **All Members:** Intensive playtesting across all 3 levels. Log bugs and refine the gameplay feel.
- **Đăng Khoa:** Integrate classic 8-bit background music and sound effects (SFX).

Week 8: Polish, Documentation & Bonus (8-10 hrs)

- **All Members:** Compile the Design Documentation.
- **Bonus Phase (If time permits):** Khải and Long begin implementing *Custom Level* features (new character skins, alternative items, or new enemy types). Đăng Khoa tests

loading these custom assets via a separate map file.

Week 9: Buffer & Delivery

- **All Members:** Final bug squashing and final code review.
- **Minh Khoa & Đăng Khoa:** Record and edit the final gameplay demo video.

4. Workflow & Source Control

(To be defined: Branching strategy, commit conventions, and pull request review process on GitHub).
Github URL: trannhukhai98438/CS202-Group03-2DGame
Branching Strategy:
The repository is structured around two main branches to separate the source code from the project documentation:

- dev ***branch:*** *The primary branch for all development and coding. Every team member must create their own feature branches (e.g.,* feature/hero-movement*,* feature/enemy-ai*) branching off from* dev*. Once a task is completed, it should be merged back into* dev *via a Pull Request.*
- docs ***branch:*** *Dedicated exclusively to project documents. This includes the design documentation, weekly reports, work division structures, and any other non-code files.*

Commit Conventions:
To maintain a clean, readable, and trackable version history, all team members must adhere to the following standard commit prefixes:

- feat: *for adding a new feature or functionality (e.g.,* feat: implement Factory pattern for items*).*
- fix: *for fixing a bug (e.g.,* fix: resolve character clipping through tilemap*).*
- docs: *for documentation updates (e.g.,* docs: add week 3 progress report*).*
- refactor: *for code changes that neither fix a bug nor add a feature, but improve code structure (e.g.,* refactor: reorganize Character class inheritance*).*
- style: *for formatting changes, removing white spaces, or adding comments that do not affect the code's logic.*

5. CMake Instructions

To compile and build the SFML C++ project, ensure you have CMake (version 3.10 or higher) and a compatible C++ compiler installed. Run the following commands in your terminal from the root directory of the project:

Step 1: Create and enter the build directory
This ensures that all compiled files are kept separate from the source code.
- mkdir build
- cd build

Step 2: Generate the build system files
This command tells CMake to look for the CMakeLists.txt file in the parent directory and configure the project.
- cmake ..

Step 3: Compile the project
This builds the actual executable based on your system's default build tool (Make, MSBuild, Ninja, etc.).
- cmake --build .

Step 4: Run the game
Once the build is successful, execute the compiled file. (Note: The exact executable name depends on how it is defined in your CMakeLists.txt).
- On Linux / macOS: ./CustomPlatformer
- On Windows (PowerShell/CMD): .\Debug\CustomPlatformer.exe